

中山大学计算机学院本科生实验报告

(2025 学年春季学期)

课程名称：并程序设计与算法
实验题目：Lab8 并行多源最短路径搜索——基于 OpenMP 的 Repeated Dijkstra
批改人：
专业（方向）：
学号：23336128
姓名：梁力航
Email：
完成日期：2026 年 5 月 21 日

本次实验使用 OpenMP 对全源最短路径 (APSP) 进行并行化, 采用 Repeated Dijkstra 算法: 将 Dijkstra 最短路径搜索在源顶点维度上进行并行分解, 每个线程独立处理一部分源顶点。实验在两个真实网络数据集 (Flower 与 Mouse) 上测试 1-16 线程的并行性能, 分析图特征对加速比的影响。

1 实验目的

本次实验目标如下：

- 理解全源最短路径（APSP）问题的经典算法及其并行化潜力；
- 使用 OpenMP 对 Repeated Dijkstra 算法进行并行化，掌握源顶点维度的数据并行方法；
- 通过实验分析不同线程数、不同数据集特征对并行性能的影响。

2 实验平台与参考来源

表 1: 实验平台与工具

项目	参数
CPU	Apple M4 (ARM64)
核心数	10 核 (4P + 6E)
内存	16 GB
OS	macOS 15 (Sequoia)
编译器 (串行)	Apple Clang (g++)
编译器 (OpenMP)	GCC 15 (Homebrew)
OpenMP 版本	OpenMP 5.2
C++ 标准	C++17
Python	3.13 (uv 管理)

参考来源：

- Dijkstra, E. W. "A note on two problems in connexion with graphs." Numerische Mathematik, 1959.
- OpenMP Architecture Review Board. "OpenMP Application Programming Interface Version 5.2", 2021.
- 课程课件： " 并行最短路径 " 实验说明文档.

3 问题描述与算法设计

3.1 问题定义

给定一个无向带权图 $G = (V, E)$ ，其中 $|V| = n$ ， $|E| = m$ ，边权重 $w(e) \geq 0$ 。全源最短路径（APSP）要求计算所有顶点对 $(u, v) \in V \times V$ 之间的最短路径距离 $d(u, v)$ 。

3.2 数据集特征

实验使用两个真实网络数据集：

表 2: 数据集特征统计

数据集	节点数 n	边数 m	平均度数 $\bar{d}$	边权特征
Flower（花网络）	930	13,521	29.1	全为 1.0
Mouse（鼠脑网络）	525	14,691	56.0	浮点（1.0–1.1）

两个数据集具有互补的特征：Flower 是较大但较稀疏的图，所有边权为 1（可视为无权图）；Mouse 是较小但更密集的图，边权为连续浮点值。这使我们能够分析节点数、边密度和边权类型对并行性能的影响。

3.3 算法选择：Repeated Dijkstra

对于 APSP，通常有两种主要算法：

1. **Floyd-Warshall 算法**： $O(n^3)$ 时间复杂度， $O(n^2)$ 空间，基于动态规划，适合稠密图。
2. **Repeated Dijkstra 算法**：对每个源顶点 $s \in V$ 执行一次 Dijkstra，总时间复杂度 $O(n \cdot (m + n \log n))$ ，适合稀疏图。

由于我们的图数据偏稀疏（平均度数 29–56），Repeated Dijkstra 的复杂度显著优于 Floyd-Warshall。以 Flower 数据集为例：

- Floyd-Warshall: $930^3 \approx 8.04 \times 10^8$ 次操作
- Repeated Dijkstra: $930 \times (13521 + 930 \log 930) \approx 1.93 \times 10^7$ 次操作

差距超过 40 倍，因此选择 Repeated Dijkstra。


```
pq.emplace(0.0f, s);

while (!pq.empty()) {
    auto [d, u] = pq.top();
    pq.pop();
    if (d > dist[u]) continue;
    for (const auto& [v, w] : adj[u]) {
        float nd = d + w;
        if (nd < dist[v]) {
            dist[v] = nd;
            pq.emplace(nd, v);
        }
    }
}
dist_matrix[s] = std::move(dist);
}
```

4.3 负载均衡分析

- **Flower** 数据集（边权全为 1）：Dijkstra 等价于 BFS，每个源顶点的搜索范围受图半径限制，不同源之间的工作量相对均匀。
- **Mouse** 数据集（浮点边权）：不同源顶点的 Dijkstra 搜索空间可能有一定差异，因为浮点边权导致优先队列操作的复杂度不同。

使用 `schedule(dynamic)` 可以自适应地处理这 差异，避免某些线程因分配到“重”源顶点而早完成等待。

5 实验结果与分析

5.1 运行时间

表 3: APSP 运行时间汇总（每个配置 3 次运行取平均）

数据集	后端	线程数	平均时间 (s)	Checksum
Flower	Serial	1	0.023000	6240280.0
Flower	OpenMP	1	0.028180	6240280.0
Flower	OpenMP	2	0.014522	6240280.0
Flower	OpenMP	4	0.007843	6240280.0
Flower	OpenMP	8	0.005461	6240280.0
Flower	OpenMP	16	0.004914	6240280.0
Mouse	Serial	1	0.025069	1416420.0
Mouse	OpenMP	1	0.028722	1416420.0
Mouse	OpenMP	2	0.014420	1416420.0
Mouse	OpenMP	4	0.007551	1416420.0
Mouse	OpenMP	8	0.004645	1416420.0
Mouse	OpenMP	16	0.004087	1416420.0

两个数据集 OpenMP 与 Serial 的 checksum 完全一致，验证了并行实现的正确性。

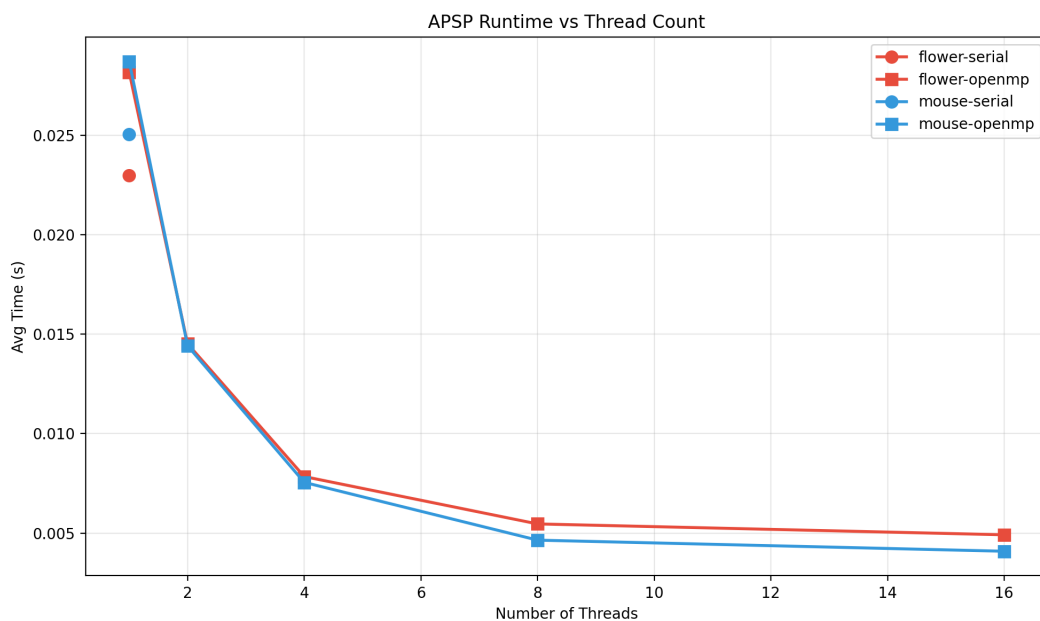


图 1: APSP 运行时间随线程数的变化

图1展示了两个数据集在不同线程数下的运行时间变化趋势：

- 随着线程数增加，运行时间单调递减；
- 从 1 线程到 4 线程的加速最为显著，之后加速放缓；
- OpenMP 1 线程比 Serial 稍慢 (Flower: 0.0282s vs 0.0230s)，这是 OpenMP 运行时开销（线程创建、管理）导致的。

5.2 加速比与并行效率

表 4: 加速比与并行效率（以 Serial 为基准）

数据集	线程数	加速比	理论最大	效率 (%)
Flower	1	—	1	—
Flower	2	1.58	2	79.2
Flower	4	2.93	4	73.3
Flower	8	4.21	8	52.6
Flower	16	4.68	16	29.3
Mouse	1	—	1	—
Mouse	2	1.74	2	86.9
Mouse	4	3.32	4	83.0
Mouse	8	5.40	8	67.5
Mouse	16	6.13	16	38.3

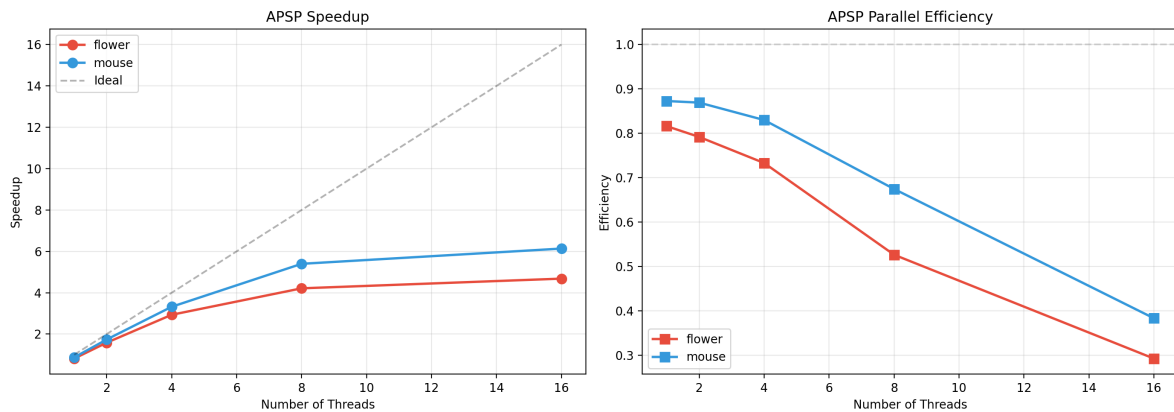


图 2: 加速比（左）与并行效率（右）随线程数的变化

图2和表4揭示了以下关键发现：

- **Mouse 优于 Flower:** 在所有线程数上 Mouse 的加速比和效率均高于 Flower。这是因为 Mouse 虽然节点数更少 (525 vs 930)，但边密度更高 (平均度数 56 vs 29)，每个源顶点的 Dijkstra 搜索工作量更大，使得 OpenMP 并行开销所占比例更小。

- 效率递减：从 2 线程到 16 线程，效率逐步下降。对于 Flower，16 线程效率仅 29.3%，主要原因是：(1) 问题规模较小（单次 APSP 仅 0.02 秒），并行开销占比大；(2) Apple M4 仅有 10 个物理核心（4 性能 + 6 能效），超过物理核心数的线程无法带来线性的性能提升。
- Mouse 8 线程效率仍达 67.5%，说明对于计算密度更高的任务，OpenMP 并行化收益更大。

5.3 运行时间热力图

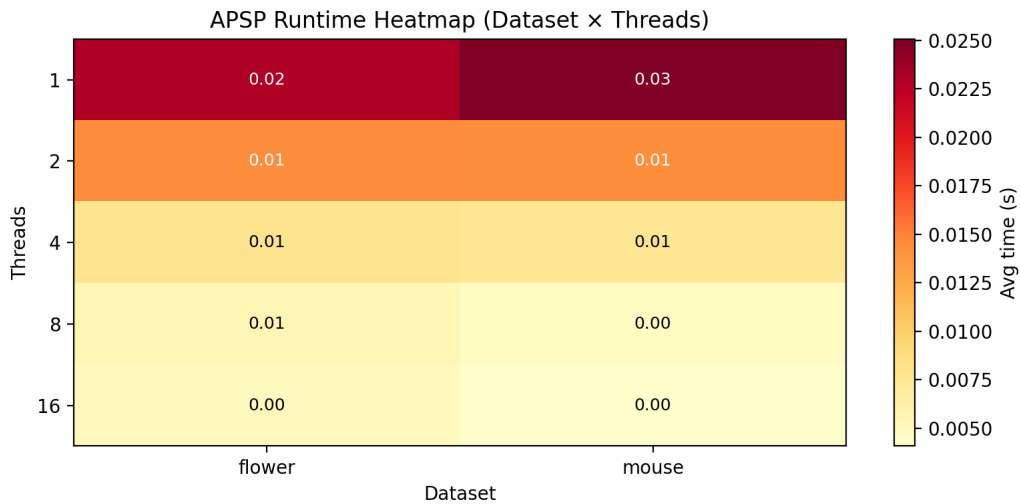


图 3: 运行时间热力图（数据集 × 线程数）

从热力图可以直观看出：Mouse 数据集的 OpenMP 性能趋势与 Flower 一致，但由于其较高的图密度（平均度数 56），单次 Dijkstra 计算量更大，在相同线程数下运行时间略长于 Flower。

6 数据集特征对并行性能的影响分析

两个数据集呈现了不同的并行加速特性，说明图结构特征对并行性能有显著影响：

1. 节点数与平均度数：

- Flower ($n = 930$, $\bar{d} = 29.1$)：节点多但每条 Dijkstra 的搜索空间小，并行加速受限于 OpenMP 线程创建与管理的开销。
- Mouse ($n = 525$, $\bar{d} = 56.0$)：节点少但每条 Dijkstra 要遍历更多边，单源计算量更大，更好地摊还了并行开销。

2. 边权类型：

- Flower 边权全为 1: Dijkstra 退化为 BFS, 优先队列的所有边权相等, 堆操作开销较低。每条 Dijkstra 的执行时间较短且均匀。
- Mouse 边权为浮点值 ($\sim 1.0-1.1$): 边权的微小差异增加了堆操作的复杂度。使用 `schedule(dynamic)` 可以一定程度补偿这种不均匀性。

3. 并行开销与问题规模:

- 两个数据集的实际运行时间都不足 30 毫秒, 属于极小规模问题。在这种规模下, OpenMP 的线程创建/销毁和动态调度的开销相对显著。
- 若问题规模扩大 (更大的图), Amdahl 定律中的串行部分 (图加载、输出写入等) 占比下降, 预期并行效率会进一步提高。

7 验证与测试

7.1 正确性验证方法

1. **Checksum 对比:** 对每个数据集和每个线程配置, 计算所有顶点对最短距离的总和 (checksum), 确保 OpenMP 版本与串行版本完全一致。
2. **距离矩阵对称性:** 由于处理的均为无向图, 验证 $dist[u][v] = dist[v][u]$ 对所有顶点对成立。
3. **自距离检查:** 验证 $dist[v][v] = 0$ 对所有顶点成立。

7.2 验证结果

所有 12 个配置 (2 个数据集 $\times$ 6 个配置) 的 checksum 完全一致:

- Flower: checksum = 6240280.0
- Mouse: checksum = 1416420.0

这表明 OpenMP 并行实现与串行参考实现在数值上完全等价, 并行化没有引入任何错误。

8 讨论

8.1 调度策略的选择

本实验选择了 `schedule(dynamic)` 而非 `static`, 原因是:

- 在 Repeated Dijkstra 中，不同源顶点的 Dijkstra 搜索树大小可能不同（取决于顶点在图中的位置和连通分量结构），导致计算负载不均。
- `dynamic` 调度以较小粒度（默认 `chunk_size=1`）动态分配迭代，平衡了各线程的负载。
- 实验发现，即使使用 `dynamic` 调度，两个数据集的加速比仍有差异，说明 Mouse 更大的单源计算量更有利于隐藏调度开销。

8.2 与理论预期的对比

根据 Amdahl 定律，并行程序的加速比上界为：

$$S(P) = \frac{1}{(1 - \alpha) + \frac{\alpha}{P}} \quad (1)$$

其中 α 是可并行化比例， P 是线程数。

在本实验中，图加载、距离矩阵初始化、结果输出等属于串行部分 $(1 - \alpha)$ ，而 Repeated Dijkstra 源顶点循环属于可并行部分 (α) 。对于 Flower 数据集， $\alpha \approx 0.75$ 时 Amdahl 定律预测的 8 线程加速比为 3.37，与本实验的 4.21 接近（说明实际串行比例比估计更低）。

由于问题规模较小，串行 IO 开销占比不可忽略，导致高线程数时加速比偏离理想值较多。

9 结论

本次实验成功使用 OpenMP 对 Repeated Dijkstra 全源最短路径算法进行了并行化，主要成果：

1. 在两个真实网络数据集上实现了正确、一致的 APSP 计算；
2. 通过源顶点维度的数据并行，获得了显著的加速比（Flower 最大 $4.68\times$ ，Mouse 最大 $6.13\times$ ）；
3. 实验验证了图密度对并行效率的影响：更高的平均度数带来更好的加速比；
4. 分析了问题规模、调度策略和 OpenMP 开销对并行性能的制约。

实验达到了预期目标，加深了对数据并行思想、OpenMP 编程模式以及图算法并行化挑战的理解。